

FRED TALKS

13th August 2026

CONTENTS

Surrender as a non-stupid life strategy

SASHA CHAPIN · 2104 WORDS

Zen of AI Coding

YOAV AVIRAM · 1912 WORDS

How agents are transforming work | OpenAI

OPENAI · 1582 WORDS

Geology Building

XKCD.COM · 12 WORDS

Surrender as a non-stupid life strategy

SASHA CHAPIN · 04 JUL 2026 · [SOURCE](#)

After getting everything I wanted in my 20s, and discovering that I was still miserable, I decided that I should no longer be in charge of choosing what I want, and I slowly settled into a frame of, “I’ll do whatever God seems to want from me.”

Some years after making that decision, I’m much happier. My life is richer, and I’m doing more for others. Regularly, I react with positive disbelief when I look at what my life has become. It looks like I’ve been executing a genius strategy. But in reality, I am living in a state of ongoing confusion, just seeing what happens.

If you’ve achieved a bunch of your goals, and you’re still not happy with your life, consider giving up on self-chosen plans. Instead, listen to your life, inner and outer, and see if you can feel a current. Then fall into it, especially if it has nothing to do with the story you had about where you were supposed to be going.

This choice is known as surrender. As with most cliches, I don’t think I can explain the profound experiential content of surrender—what it feels like, why it works. But perhaps I can lay out a constellation of propositions^[1] that will, together, illuminate a worldview very different from the one I once inhabited.

Knowing and not-knowing yourself

1.1 The planning part of your mind, the one that generates ambitions and hypothetical situations, is a small part of your overall intelligence. Anyone who is halfway good at sex, comedy, or any creative art will know this. Designing your life based on what that part of your mind wants is like designing your house to fit the needs of your right hand. Congratulations, your house is filled with fidget spinners and soap.

1.2 A large percentage of human suffering is anticipatory tension, or dreading a future experience such that you actualize its potential suffering in the now. Anticipatory tension is using the illusion of knowledge to generate the illusion of control. Much of what people call planning is this.

1.3 My best outfits are all selected by someone else. When I'm at a clothing store, and I meet an employee who seems engaged, I tell them to dress me for maximum attractiveness. They do a much better job than I do. Of course: I can't see myself as well as someone else can. This is why I try to listen to what the world apparently wants to do with me, rather than who I imagine I am.

1.4 Often, our self-image is a defense against the traits we reject in ourselves. "I love to be spontaneous," says the most calculating person you've ever met. Thus, if you're trying to live in self-loathing and denial, one way is to sculpt your life around the self-image you've decided on.

1.5 I've discovered that scrutinizing my motives too closely is a great way to stop creating. The unexamined life may not be worth living, but the fully examined life is a pinned butterfly in a glass case. I've concluded that my life is a grand artistic project, but I'm best off not knowing what it is.

1.6 You know that person who had important ripple effects in your life, based on a brief meeting or a kind gesture they extended to you, which meant nothing to them, but everything to you? Or that interesting stranger whose manner cast a brief spell on you, lingering in your memory disproportionately? That's you also, you're that person. Most of your impact on others, you will never fully apprehend or appreciate. Thus, it's not important to know what you're doing.

1.7 The proper end of introspection is not being filled with facts about oneself. Your introspection is successful if it is finite, if it leads you towards direct contact with experience. Get out of the way of your becoming.

Choice and authenticity

2.1 People feel most themselves when they are unselfconscious, which is to say, their actions emerge without apparent control. Think of the phrase "singing with abandon." In our most profound moments of intimacy, we have abandoned the wheel. Who is driving, at that moment? What if we simply let that driver have the wheel more often? It takes practice to notice when that's the person taking over the wheel—as opposed to our reactive, fear-based conditioning, which is also eager to take the job. Surrender is a skill.

2.2 When people talk about "being authentic," they are often referring to executing some program, some persona, that they've devised privately. Of course, our impression of such people is that they're unnatural and unoriginal. They assign us the job of reinforcing a character we never agreed on.

2.3 Surrender is something that happens through you, not something chosen^[2]. I couldn't have skipped the part of my life when I was contriving everything. And the way I'm living now wouldn't work without the skills I accumulated during that period of contrivance. If you're reading this and thinking, "gosh, I wish person x would see the wisdom of surrender," you have discovered an opportunity to surrender to interpersonal helplessness.

2.4 Often, the apparent struggle to make a decision is an attempt to meet an internal flailing quota, after which we will feel affirmed in the choice we already knew we were making. Sometimes you can just skip all of that, and admit that you're scared of what you know you will do. And then do it.

Goals, outcomes, effort

3.1 Discovered goals are entirely different from preconceived goals. Let's say you notice that when you play guitar and sing, people stop what they're doing to stare at you. The discovered goal of being a musician has presented itself. You are more likely to be successful than someone who privately dreams of being a musician without prompting from existence. Noticing such opportunities requires a lack of some previously decided agenda.

3.2 I've known many impressive people. Almost none of them started with a comprehensive vision of the grand work they would produce. Instead, they simply got into escalating amounts of trouble, and kept riding the trouble wave as far as it would go.

3.3 Excellence requires non-attachment to principles. All mediocre performers are the same: instead of improving on feedback from what happens, they react to what they think is supposed to be happening, or try to reinforce a theory they've internalized—perhaps an inaccurate approximation of someone they admire. This attempt at coherence is futile, since the mind of an expert makes no sense. It is filled with exceptions, contradictory principles butting up against each other. Perhaps the knowledge of how your life works is the same.

3.4 When a project in my life is working, it's typically marked by a sense of leverage. It's as if I've fallen into an updraft and it's propelling me. Meanwhile, when I'm privately excited about something, but there's no cooperation from the world, nothing comes together. The sensible conclusion is that my enthusiasm, alone, is not a helpful measurement.

3.5 It's nice to enjoy what you're doing. But there is a greater liberation available: accepting that the Work in front of you is your Work, and that has little to do with how you feel.

3.6 Another property of things that work out for me: they don't make sense to my rationalizing mind. There is the thought, "this isn't what I'm supposed to be doing," even as greater forces pull me forward. This is known as the refusal of the call to adventure. It's a routine part of adventure.

3.7 There is a criticism you could make of my advocacy of surrender. It is: "you're just lucky and narrativizing it." Yes. If you, too, have luck that seems unrelated to your conscious choices, consider letting that run the show.

Happiness

4.1 The norm for human life: gross injustice, corrupt rulers, disease and death, unequal fortunes, looming apocalypses. It's possible that a large portion of our species will die off—it's happened before, we've almost gone extinct previously, that's allowed in the game. If your basic needs are met but you cannot be content with the way it is now, that is not the fault of the way things are now. It's because you haven't surrendered to the basic properties of incarnating in human form.

4.2 Strangely, happiness isn't produced by getting what you want. Unhappiness is the product of the number of times per minute that you believe circumstances should be otherwise. "Enough" and "not enough" are mental stances that, very often, are uncorrelated with exterior facts. Perhaps this is one reason Jesus says it is "easier for a camel to go through the eye of a needle than for a rich man to enter the kingdom of God"—the more you focus on expanding your pile, the more ravenousness becomes a habit. By contrast, you can, right now, try believing that what's happening is what's supposed to be happening, in exactly the right order.

4.3 Satisfaction comes from giving yourself away. From realizing that you have been called to some occasion that requires everything from you, and saying, fine, fuck it, here I am, tear me apart, I will exhaust myself to set right this one irrelevant corner. It is possible to view your whole existence this way, as a process of giving away all, everything, of what was never yours.

4.4 Another profound source of satisfaction is fit: when you fit like a slender hand in the glove of your circumstances. Consider the possibility that this satisfaction is already trying to happen, but you are resisting it, in a doomed attempt to be happy. Settling into this fit is another way of describing what I mean by "doing what God seems to want."

4.5 I can't defend this intellectually, but it has been true in my experience: spontaneous compassion is the approach to ethics that feels best and works most often. If you want a plan for consistently being good, the dumbest one that will work, and therefore the most robust, is

“notice that prosocial behavior is nourishing, and that peace is only available if you’re on good terms with your conscience, and allow behavior to flow from that.” I’m not deluded enough to think this is always sufficient, but it’s a good start.

Love

5.1 I am capable of privately spoiling my wonderful marriage at any time, by noticing that it’s not the relationship I would’ve imagined being in. Meanwhile, a few months before meeting my wife, I wrote down a list of characteristics that the perfect partner would have, and I subsequently met that person. She was completely lovely—I find much to criticize in my own behavior from that period and nothing bad to say about her. And yet the relationship did not stick. How odd.

5.2 When I insist that a relationship must survive, I begin to resent it—I become burdened by all of the little moments of inauthenticity engaged in to shield the other person. The person becomes an obstacle, someone to navigate around. When I accept that any relationship could end, and surrender to the reality of people finding their own level, I become free to relate openly, and then actual closeness happens.

5.3 Panic is a part of love. I’ve learned that as I enter into any relationship that will alter me for the better, the manager in my head will come up with convincing reasons why it’s all wrong. This, too, is the refusal of the call; it is the fear of personal transformation masquerading as reason.

5.4 The older I get, the more that friendship is a discovered property. There are lots of wonderful people I meet who aren’t my friends. And then I meet someone and think: oh, that’s my friend, you’re coming with me. One of the best things you can do for yourself is to notice when you’re inexplicably bound to someone, and act on it without thinking.

5.5 We are drawn towards people because they are different. We are given the opportunity to love more when we find ourselves resenting them for the very difference that drew us to them. The opportunity is to release resentment and instead surrender to separateness. Ironically, this is how you feel less alone.

Thank you to Erin Shetron for editorial assistance. This essay was inspired a little bit by The Surrender Experiment, and a lot by [this totally wonderful meditation guide](#). Photo credit goes to Daido Moriyama.

This format is shamelessly stolen from [this fascinating essay](#) by “de Pony Sum.”

[2 \[find in text\]](#)

I first heard this point made by Adyashanti.

...

Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · [SOURCE](#)

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to [Zen of Python](#) by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead
2. Code is cheap
3. Refactoring easy
4. So is repaying technical debt
5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack
8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world

11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents
16. Anticipate modes of failure

Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

All bugs are shallow

Linus's law states that “given enough eyeballs, all bugs are shallow.” Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How “dense” are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We've built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is trickier then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

Agent-first software engineering

We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.

Agent-first product strategy

I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of [WhiteBoard](https://whiteboard.it), where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboard.it.

How agents are transforming work | OpenAI

OPENAI · 28 JUN 2026 · [SOURCE](#)

June 25, 2026

Company

A new Economic Research paper measuring Codex's economic potential at the frontier.

Read the paper

Listen to article

[AUDIO OMITTED]

Agentic AI changes the unit of knowledge work from single interactions to delegated, long-horizon tasks. Chatbot interactions are often short and self-contained. Agents can operate independently for minutes or hours while orchestrating tool calls, interacting with environments, and iterating towards solutions. As a result, agents are quickly becoming the most powerful AI tool for work.

Over the last year, we witnessed this transformation first-hand at OpenAI. For the first few months after Codex was released to the public, ChatGPT remained the default AI tool for work within OpenAI. Through August 2025, the average OpenAI worker spent less than 10% of their tokens on Codex. Now, every department, including non-technical departments such as Legal and Recruiting, uses Codex as their primary AI tool for work. This pattern reflects what we believe will be the future of work given the expanded capabilities and accessibility of agentic tools.

Some content could not be imported from the original document. [View content ↗](#)

Some content could not be imported from the original document. [View content ↗](#)

Codex adoption grew in tandem with Codex's capabilities. As Codex leveraged stronger models and new product features, it became capable of taking on an expanding set of productive tasks. Across Individual users, Organizational users, and OpenAI workers, we document four trends over the past year:

- **People use Codex for longer-horizon work.** By May 2026, 80.6% of sampled individual users made at least one Codex request estimated to exceed 30 minutes of human work, 70.2% made one estimated to exceed one hour, and 25.6% made at least one Codex request estimated to exceed eight hours.
- **Codex became the primary AI tool for every department at OpenAI.** Engineering moved first, but Legal, Finance, and Recruiting crossed into Codex being their primary AI tool around April 2026. For the average OpenAI worker, Codex usage now accounts for more than 85% of output tokens. Since Codex users tend to use more tokens than non-users, its share of overall tokens is even higher: Codex accounts for 99.8% of weekly output tokens generated within OpenAI.
- **Non-developer adoption grew especially rapidly, outpacing developer adoption.** Since August 2025, non-developer users rose 137x for individual users, 189x for organizational users, and 12x within OpenAI.

- **Codex enabled OpenAI workers to do tasks outside their job description.** While technical usage is still most prevalent among engineers, non-technical users regularly use Codex to take on coding or technical execution, including automation, data transformation, tooling, debugging, and structured analysis.

Agents work longer hours on harder tasks

Nearly a quarter of all Codex requests are for tasks that would take a person more than one hour to complete⁽¹⁾. As Codex's capacity for independent long-context work improved, users shifted from short interactions toward more difficult tasks with longer horizons.

The chart below estimates the share of individual users that crossed four human-time thresholds: tasks that would take a person more than 30 minutes, more than one hour, more than four hours, and more than eight hours⁽²⁾. From December 2025 to May 2026, the share of users who made a request that was estimated to correspond to work that would take a person more than 30 minutes rose to 80.6%. The share making a request that would take a person more than one hour rose to 70.2%. The share requesting work that would take a person more than eight hours grew the fastest from a low base.

The growth in agentic usage can be seen in daily Codex runtime. Among daily active users at OpenAI, the heaviest users ask Codex to run many hours of agent work in a single day. By June 2026, users at the 99th percentile regularly generated more than 60 hours of Codex agent turns per day, distributed across multiple, parallel agents. As Codex became more powerful and parallelizable, users moved from only asking Codex for one answer at a time into increasingly orchestrating multiple agent tasks over the course of a day.

Adoption continues to move from engineers to the rest of OpenAI

Engineers at OpenAI began adopting Codex first, gradually. The average engineer at the company shifted the majority of their usage of OpenAI products to Codex by December 2025. Today, the average engineer generates 99% of their output tokens with Codex rather than ChatGPT. Legal, finance, and recruiting crossed over to majority use of Codex later, around April 2026, but their transitions were much faster. The average lawyer or recruiter at OpenAI now generates more than 85% of their output tokens on Codex.

Over the last six months, Codex usage has deepened and intensified at OpenAI. Among active internal users, change in combined output tokens rose sharply across departments. Research saw the biggest jump: by June 2026, median use was 56 times higher than in November 2025. Customer Support rose 32 times and Engineering rose 27 times, while Legal grew more gradually but still reached 13 times its November level.

These two patterns together illustrate how Codex has transformed how OpenAI uses AI to do productive work. Across the company, users are switching from chatbots to agents as their primary form of AI interaction and are deploying an exponentially growing amount of agentic labor.

Non-developers are the fastest-growing user groups

Across all user groups—OpenAI, organizational, and individual users—Codex use began with developers, the natural target audience for what began as a coding tool. However, as Codex expanded toward more general knowledge work, adoption among non-developers grew even more quickly. As shown in the user growth chart below, weekly non-developer users rose faster than developer users among individual, organizational, and OpenAI populations. By early June 2026, non-developer individual users multiplied 137 times since August 2025. Non-developer organizational users increased 189-fold, and non-developer OpenAI users increased 12-fold, likely because this group already started at a well above average starting point.

The shift does not mean every non-developer is using Codex in the same way as an engineer. Rather, it means that more non-developers are using Codex for some kind of agentic work.

Codex is expanding the horizon of potential productive work

Codex enables non-technical departments to accelerate workflows previously bottlenecked by technical expertise. The heat map below compares inferred occupations within OpenAI to the type of work represented in Codex outputs. Engineering and coding show up as the largest category for data science and research, whereas knowledge work is the largest category for finance and business operations, marketing, operations, and other departments.

That said, agentic tools can expand what an individual worker can do. For instance, over one-fourth of work done with Codex by workers in business functions was engineering or coding. Agents can lower the cost of moving across task boundaries and help workers do adjacent work that used to require more specialized technical support.

Occupation vs. work done with Codex

Consolidated inferred department	Work category				
	Engineering / Coding	Data Analysis	Financial Analysis	Knowledge Work	Other
Engineering	72%	4%	1%	18%	5%
Data Science / Research	51%	10%	0%	30%	9%
Finance / Biz Ops	31%	9%	16%	34%	10%
Product / Marketing / Ops	25%	3%	7%	51%	15%
Other	50%	7%	2%	38%	4%

Share of output tokens within department

What this means for agents’ economic potential

Increased use of agentic tools by non-engineer employees expands the frontier of what these workers can do. That matters for businesses deciding how to redesign workflows, for employees learning which skills become more valuable, and for policymakers and researchers trying to understand how AI changes the labor market.

Our paper shows how frontier users adopt capable agentic tools at the frontier. Our results demonstrate what unfolds when people have broad, low-friction access to capable agentic tools: as the tools improve, people use them for longer, more complex, and more cross-functional work. As time goes on, this is likely to be what the future of work looks like.

- [2026](#)
- [Economic Research](#)
- [Codex](#)

Author

OpenAI

Footnotes

1. Task horizon is estimated using an LLM-as-judge with access to Codex transcripts.

[SVG OMITTED]

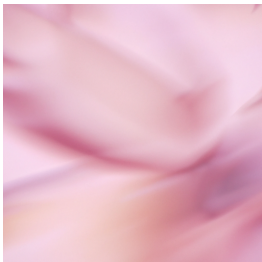
[find in text]

2. These thresholds are model-estimated, so they should be treated as directional rather than exact. This is also based only on individual data and based on the queries of a random sample of 0.1% of users. [SVG OMITTED]

[find in text]

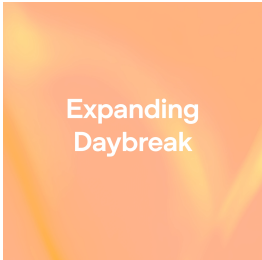
Keep reading

[View all](#)



[OpenAI and Broadcom unveil LLM-optimized inference chip](#)

[CompanyJun 24, 2026](#)



Daybreak: Tools for securing every organization in the world

SecurityJun 22, 2026

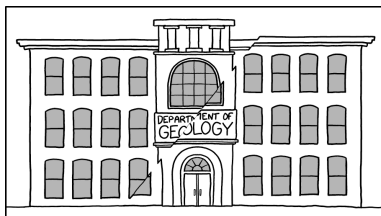


Samsung Electronics brings ChatGPT and Codex to employees

CompanyJun 21, 2026

Geology Building

XKCD.COM · 13 AUG 2026 · [SOURCE](#)



THE ADMINISTRATION REGRETTED GIVING THE GEOLOGY
DEPARTMENT FREE REIGN TO DESIGN THEIR NEW BUILDING.

The administration thinks it's weird, but the geologists say
it looks normal.